Video 05 — Program execution

Read only the narration paragraphs. Section headings, numbers and timestamps are not spoken. Timings refer to the original edit; speak naturally.

01. Where does seven go? | 00:00–00:33

Three deliveries, plus four. Where does seven go?

Pip's tiny demonstration program must add two values and save the result.

The processor follows instruction meanings and changes stored state. It does not understand the whole program like a reader understands a story.

Have you ever wondered how a line of code becomes a visible result?

Chapter one, The Fundamentals, lets us trace the registers and follow every important state change.

02. Today’s route | 00:33–01:11

You will explain fetch, decode, and execute, and give each teaching-model register a clear job.

You will trace four instructions and distinguish an immediate value from a memory address.

You will also connect source code, translators, and machine instructions without oversimplifying how modern language implementations work.

Welcome to the tutorial on program execution by Blue Impulse.

Have a small table ready with columns for the program counter, current instruction, accumulator, and memory location two hundred.

03. Before the processor executes | 01:11–02:01

The subtopic From source code to execution explains that machine code contains the instructions a processor executes.

An assembler translates assembly mnemonics and labels. A compiler produces another representation, often native code or bytecode.

An interpreter runs operations through a runtime, sometimes working with bytecode.

The same language can have compiled and interpreted implementations, and modern systems may combine several techniques.

Avoid the blanket claim that every interpreter translates exactly one source line at a time.

Our demonstration uses invented assembly notation so that every state change remains easy to see.

04. Use the recipe analogy | 02:01–02:51

Use a recipe to remember the register jobs. The program counter marks the next recipe line.

The memory address register selects the location. The memory data register carries the item read from memory.

The current instruction register holds the instruction being followed. The accumulator is a temporary mixing bowl for arithmetic.

We abbreviate those names as P C, M A R, M D R, C I R, and A C C.

The analogy helps you remember the jobs, but these are electronic storage locations inside a processor model.

Keep the jobs distinct: an address points to a location, while an instruction or value is the content being handled.

05. Fetch one instruction | 02:51–03:54

Follow the fetch sequence. First copy the address in the program counter to the memory address register.

Read the instruction at that memory location into the memory data register.

Advance the program counter so it identifies the next instruction address, then copy the fetched instruction into the current instruction register.

The control unit can now decode the operation and its operand, and the processor executes the required work.

In this simplified model, every instruction occupies one word, so the program counter advances by one.

Real instruction sizes and exact timing depend on the architecture. Our table is a teaching model, not every processor's wiring diagram.

An address identifies a place. A value is the information stored or processed there. Keeping these roles separate makes the register trace much easier to follow.

06. Read the four instructions | 03:54–04:42

In the subtopic Trace a tiny program, the first instruction sits at address one hundred: LOAD, hash three.

The hash sign denotes an immediate value in this invented notation, so that instruction puts the number three into the accumulator.

Address one hundred and one contains ADD, hash four. Address one hundred and two contains STORE, two hundred.

Address one hundred and three contains HALT, which ends this teaching program.

The number two hundred in STORE identifies a memory location. It is not the value to put into the accumulator.

Before running anything, predict what that memory location will contain when the program finishes.

07. Single-step the worked program | 04:42–05:51

Let's single-step. Initially, the program counter is one hundred. Fetch the LOAD instruction and advance to one hundred and one.

Decode and execute LOAD, hash three. The accumulator now contains three.

Next fetch ADD, hash four from address one hundred and one. The program counter advances to one hundred and two.

Execution adds four to the accumulator's three, giving seven. Memory location two hundred has not been written yet.

Fetch STORE, two hundred. The program counter advances to one hundred and three, and execution copies seven into memory location two hundred.

The accumulator still contains seven. STORE copies the value; it does not turn the accumulator into an address.

Finally fetch HALT. In this simplified trace, the fetch advances the counter to one hundred and four before execution stops.

The runnable Python model prints these same states and register transfers, so you can reproduce the trace yourself.

08. Value or address? | 05:51–06:44

Pause and check your prediction. After the STORE instruction, what are the accumulator and memory location two hundred?

Also ask yourself whether STORE, two hundred means storing the number two hundred.

The accumulator remains seven, and memory location two hundred receives seven. Two hundred is the destination address.

The hash sign in our LOAD and ADD operands marks literal values. The STORE operand has a different role.

This distinction prevents a common tracing mistake: confusing where data lives with the value stored there.

Use this model's instruction meanings before applying assumptions from a different programming language or processor.

09. The next address can change | 06:44–07:32

A branch instruction may replace the normal next value of the program counter.

That is how the instruction sequence can follow a decision or repeat a loop instead of always moving forward.

Our four-instruction program contains no branch, so the normal next addresses stay easy to follow.

Chapter three, Hardware, adds pipelining: real processors may overlap instructions while preserving the required program behaviour.

Overlapping the implementation does not make the logical effects of a program arbitrary.

For a basic trace, first understand the defined instruction effects, then study how a specific processor implements them.

10. Change one instruction | 07:32–08:26

Your turn. Change ADD, hash four to ADD, hash nine, while keeping LOAD, hash three and STORE, two hundred.

What value should reach memory location two hundred? Work through the accumulator before we reveal it.

LOAD gives three. Adding nine gives twelve. STORE copies twelve into memory location two hundred.

That is a small example of testing a prediction by changing one input to the program.

The included simulator accepts a different add value, prints the trace, and checks that the final memory result matches the arithmetic.

Try another value and explain each register's role aloud as the instruction moves through the cycle.

11. Instructions change machine state | 08:26–09:32

Recap the lesson. Source code reaches execution through a translator or runtime appropriate to its implementation.

The program counter identifies the next instruction. Fetch brings it from memory, decode interprets it, and execute performs its operation.

The memory address register selects a location, the memory data register carries data, and the current instruction register holds the operation.

Our teaching accumulator went from three to seven, and STORE copied seven to memory location two hundred.

Addresses and values have different roles. Branches can change the normal next instruction address.

Complete the trace without watching the highlight. Understanding means predicting each state change yourself.

Pip's seven is now a stored result because STORE copied the accumulator value to the chosen memory address. The instruction determined both what changed and what stayed the same.

12. Pip completes the shift | 09:32–10:00

Pip's small problem now has an explanation you can test.

Use Chapter 1, The Fundamentals, and the named subtopics to revisit the reasoning.

The tutorial guide includes the full teaching steps and the checked practice answers. Run the supporting demonstration and change one input.

Blue Impulse. Follow the signal, predict the result, and check your explanation.